

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85 };
86
87 //create a modified fn with some additional logic
88 var modifiedFn = function() {
89
90     //mark the element as visible
91     t.appeared = true;
92
93     //is this supposed to happen only once?
94     if (settings.one) {
95
96         //remove the check
97         w.unbind('scroll', check);
98         var i = $.inArray(check, $.fn.appear.checks);
99         if (i >= 0) $.fn.appear.checks.splice(i, 1);
100     }
101
102     //trigger the original fn
103     fn.apply(this, arguments);
104
105     //bind the modified fn to the element
106     //if (settings.one) t.one('appear', settings.data, modifiedFn);
107     t.on('appear', settings.data, modifiedFn);
108 };

```



MATERIA:

Programación Orientada a Objetos

Unidad 3 Programación Orientada a Objetos en Desarrollo Moderno

Docente:

Grupo:

Unidad #3 - Programación Orientada a Objetos en Desarrollo Moderno.

3.7 Métodos HTTP: GET, POST, PUT, DELETE.

3.8 Introducción a @RequestBody, @PathVariable, @RequestParam.

3.9 Uso de DTOs (Data Transfer Objects) para entrada/salida de datos.

3.10 Validación básica con @Valid y @NotNull.

3.7: Métodos HTTP - Las Órdenes del Mesero

¿Qué son los Métodos HTTP?

Son los verbos de nuestras peticiones. Le dicen al servidor la intención de lo que queremos hacer.

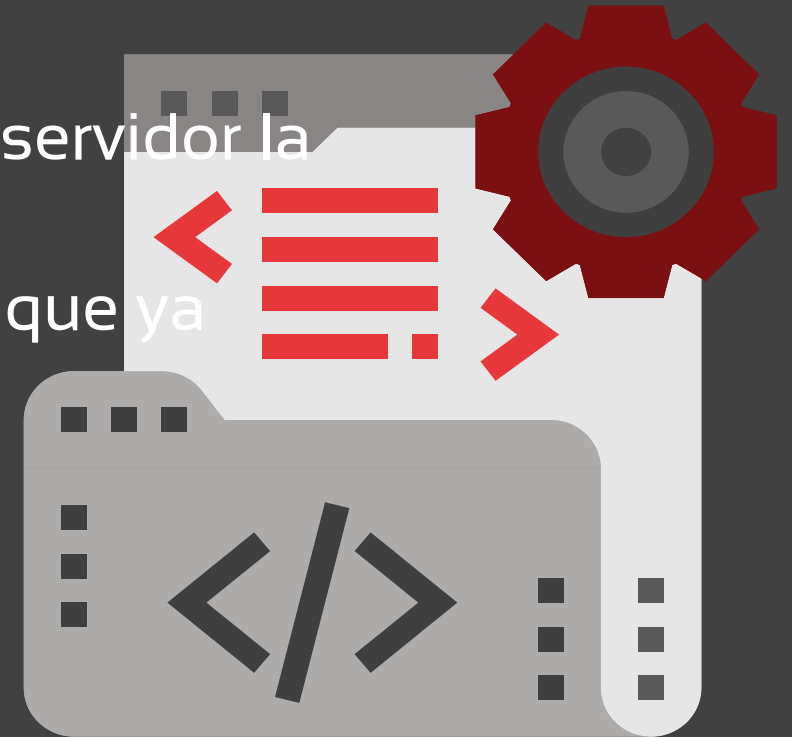
Se mapean directamente a las operaciones CRUD que ya conocemos.

Create -> POST

Read -> GET

Update -> PUT

Delete -> DELETE



3.8 Anotaciones - ¿De Dónde Vienen los Datos?

Spring necesita saber de dónde tomar los datos que el cliente envía. Usamos 3 anotaciones clave.

- **@PathVariable**: De la ruta de la URL.
- **@RequestParam**: De los parámetros de consulta (?).
- **@RequestBody**: Del cuerpo de la petición.

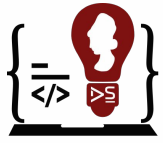
3.9 DTOs - La Comanda Simplificada

El Problema de Exponer Entidades

Tu clase `@Entity` puede tener:

- Datos internos que no quieres mostrar (`internalAuditTimestamp`).
- Relaciones complejas que causan bucles infinitos.
- Demasiada información que el cliente no necesita.

¡Nunca expongas tus entidades directamente!



3.10 Validación - El Mesero Revisa tu Pedido

¿Porqué?

Para evitar que datos incorrectos o maliciosos lleguen a tu sistema.

- Nombres vacíos.
- Precios negativos.
- Emails sin formato correcto.

Regla: "Nunca confíes en los datos que envía el cliente".

Usamos anotaciones de `jakarta.validation` en el DTO de entrada.

Java

```
public class CreateProductDTO {  
  
    @NotNull(message = "El nombre no puede ser nulo")  
    @Size(min = 3, max = 50)  
    private String name;  
  
    @NotNull  
    @Positive(message = "El precio debe ser positivo")  
    private Double price;  
  
}
```

En el controlador, simplemente añade la anotación `@Valid` antes del `@RequestBody`.

Java

```
@PostMapping  
public ProductDTO createProduct(@Valid @RequestBody CreateProductDTO dto) {  
    // Si la validación falla, Spring automáticamente  
    // devuelve un error 400 Bad Request.  
  
    // Si la validación pasa, el código continúa.  
    // ...  
}
```

¡Así de fácil! Tu código se mantiene limpio y solo procesa datos válidos.